

Extra: How to write a Makefile

A guide for those who want more. You do not need it for the tasks: every task folder already has a Makefile. But one day you will write your own, and then this guide helps. The examples are in [Makefile_guide/examples](#); everything below was run with GNU Make 4.3.

Why make?

You can always compile by hand:

```
g++ -std=c++20 -O0 -Wall -pthread tickets.cpp -o build/tickets
```

This is long, and in a bigger project you have many such commands. You do not want to type them, and you do not want to run all of them after every small change, because a big project compiles for minutes. `make` solves both problems:

- you write the commands **once**, in a file called `Makefile`;
- `make` runs **only** the commands that are needed, because something has changed since the last time.

The first rule

A Makefile is a list of **rules**. A rule says: “to make this file (the **target**), you need these files (the **prerequisites**), and these are the commands (the **recipe**)”.

The diagram illustrates a Makefile rule for creating a target named 'hello'. The rule is written as 'hello: hello.cpp' followed by a new line starting with a tab character and the command 'g++ -std=c++20 -Wall hello.cpp -o hello'. Annotations include: 'target' with an arrow pointing to 'hello:', 'prerequisites' with an arrow pointing to 'hello.cpp', and 'recipe' with an arrow pointing to the command. A note 'a TAB, not spaces' points to the tab character at the start of the command line.

⚠ The recipe starts with a TAB

Every line of a recipe must start with a **TAB** character, not with spaces. With spaces, `make` stops with: `Makefile:2: *** missing separator. Stop.` Most editors can show TABs; in VS Code the bottom bar shows “Spaces” or “Tab Size”.

The whole Makefile of the first example (`examples/1_one_file`):

```
hello: hello.cpp
    g++ -std=c++20 -Wall hello.cpp -o hello
```

Run `make` in this folder:

```

$ make
g++ -std=c++20 -Wall hello.cpp -o hello
$ make
make: 'hello' is up to date.
$ touch hello.cpp
$ make
g++ -std=c++20 -Wall hello.cpp -o hello

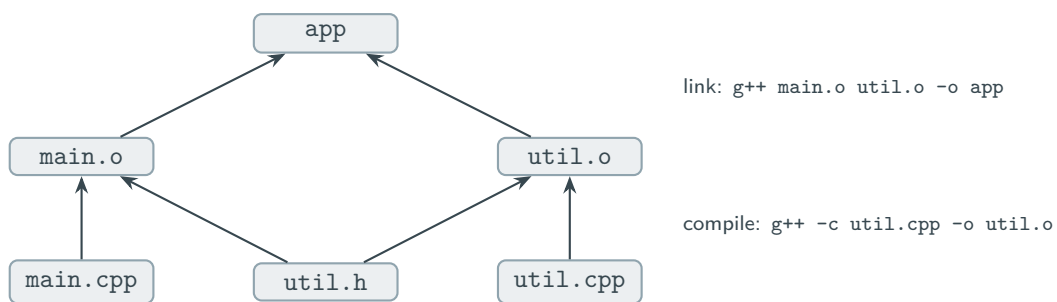
```

How does make decide? It compares the **modification times** of the files. If the target does not exist, or one of the prerequisites is newer than the target, it runs the recipe. Otherwise the target is up to date and nothing happens. `touch` only changes the time of a file, as if you had saved it.

Without arguments, make makes the **first** target in the file. You can name another one: `make hello`.

Several files: a graph of dependencies

A real program has several `.cpp` files. Each one is compiled separately into an **object file** (`.o`, the option `-c`), and then the object files are **linked** into the program. In `examples/2_three_files`, `main.cpp` calls `square()` from `util.cpp`, and both include `util.h`:



An arrow means “is needed to make”. The Makefile writes this graph down, one rule per box:

```

app: main.o util.o
    g++ main.o util.o -o app

main.o: main.cpp util.h
    g++ -std=c++20 -Wall -c main.cpp -o main.o

util.o: util.cpp util.h
    g++ -std=c++20 -Wall -c util.cpp -o util.o

```

Now `make` rebuilds only what depends on the changed file:

```

$ make
g++ -std=c++20 -Wall -c main.cpp -o main.o
g++ -std=c++20 -Wall -c util.cpp -o util.o
g++ main.o util.o -o app
$ touch util.cpp; make           # only util.o and app
g++ -std=c++20 -Wall -c util.cpp -o util.o
g++ main.o util.o -o app
$ touch util.h; make             # util.h is needed by both
g++ -std=c++20 -Wall -c main.cpp -o main.o
g++ -std=c++20 -Wall -c util.cpp -o util.o
g++ main.o util.o -o app

```

⚠ Do not forget the headers

If `util.h` were missing from the prerequisites, a change in `util.h` would not recompile anything, and the program could use an old, wrong version of the header. Always list the headers that a file includes.

Variables and automatic variables

The same compiler and the same options appear in several rules. Put them into **variables**, so that you change them in one place. `CXX` and `CXXFLAGS` are the usual names for the C++ compiler and its options.

```

CXX      := g++
CXXFLAGS := -std=c++20 -Wall -O2

app: main.o util.o
    $(CXX) $^ -o $@

main.o: main.cpp util.h
    $(CXX) $(CXXFLAGS) -c $< -o $@

util.o: util.cpp util.h
    $(CXX) $(CXXFLAGS) -c $< -o $@

```

<code>NAME := value</code>	define a variable; <code>\$(NAME)</code> uses it
<code>\$@</code>	the target of this rule, here <code>main.o</code>
<code>\$<</code>	the first prerequisite, here <code>main.cpp</code>
<code>\$^</code>	all prerequisites, here <code>main.o util.o</code>

A variable can be changed from the command line, without editing the file. `-B` makes everything again, even if it is up to date:

```

$ make -B CXXFLAGS="-std=c++20 -O0"
g++ -std=c++20 -O0 -c main.cpp -o main.o
g++ -std=c++20 -O0 -c util.cpp -o util.o
g++ main.o util.o -o app

```

:= or =?

`A := $(B)` copies the value of `B` at this moment. `A = $(B)` remembers “the value of `B`” and reads it each time `A` is used, so a later change of `B` also changes `A`. When in doubt, use `:=`.

Pattern rules and .PHONY

The rules for `main.o` and `util.o` are the same, only the name differs. A **pattern rule** with `%` writes them once: “any `X.o` is made from `X.cpp`”. And some targets are not files at all, but names of actions: `all`, `run`, `clean`. Mark them as `.PHONY`, so that `make` runs them even if a file with this name happens to exist.

```
CXX      := g++
CXXFLAGS := -std=c++20 -Wall -O2
OBJ      := main.o util.o

.PHONY: all run clean

all: app

app: $(OBJ)
    $(CXX) $^ -o $@

%.o: %.cpp util.h
    $(CXX) $(CXXFLAGS) -c $< -o $@

run: app
    ./app

clean:
    rm -f app $(OBJ)
```

```
$ make run
./app
7 * 7 = 49
$ make clean
rm -f app main.o util.o
$ make -n          # only print the commands, do not run them
g++ -std=c++20 -Wall -O2 -c main.cpp -o main.o
g++ -std=c++20 -Wall -O2 -c util.cpp -o util.o
g++ main.o util.o -o app
```

The Makefile of our tasks, line by line

Every task folder has this file. Each `.cpp` file in the folder is a separate program, so there are no object files here: each `.cpp` becomes one program in `build/`.

```

# Compiles every .cpp file of this folder into build/.
# make compile
# make run compile and run
# make clear remove build/
CXX := g++
CXXFLAGS := -std=c++20 -O0 -Wall -pthread
BUILD_DIR := build
PROGRAMS := $(patsubst %.cpp,$(BUILD_DIR)/%, $(wildcard *.cpp))
# the one program of this task, for make run
PROGRAM := $(BUILD_DIR)/$(basename $(wildcard *.cpp))

.PHONY: make run clear

make: $(PROGRAMS)

$(BUILD_DIR)/%: %.cpp $(wildcard *.h)
    mkdir -p $(BUILD_DIR)
    $(CXX) $(CXXFLAGS) $< -o $@

run: $(PROGRAM)
    ./$(PROGRAM)

clear:
    rm -rf $(BUILD_DIR)

```

line	what it does
CXXFLAGS := -std=c++20 -O0 ...	the options of the compiler; -pthread is needed for threads
\$(wildcard *.cpp)	all .cpp files in the folder, for example tickets.cpp
\$(patsubst %.cpp,\$(BUILD_DIR)/%,...)	replace the name: tickets.cpp becomes build/tickets
make: \$(PROGRAMS)	the first target, so plain make builds all programs. It is called make, as in the Makefiles of Lecture 6
\$(BUILD_DIR)/%: %.cpp \$(wildcard *.h)	a pattern rule: build/tickets from tickets.cpp; it also depends on all headers, so a change in tickets.h recompiles it
mkdir -p \$(BUILD_DIR)	create build/ if it does not exist yet
\$(basename ...)	remove the extension: tickets.cpp becomes tickets, so PROGRAM is build/tickets, the one program of the task
run: \$(PROGRAM)	build the program if needed, then run it
clear	remove build/

Good to know

- `make -j8` runs up to 8 recipes at the same time, on 8 cores. It works because make knows the graph: it never starts a recipe before its prerequisites are ready.
- Every line of a recipe runs in its own shell. `cd build` on one line does not change the folder of the next line; write `cd build && ...` on one line.
- A comment starts with `#`. A long line can continue on the next one after a `\`.

- In big projects, people often let a tool write the Makefile for them (CMake, Meson). But what that tool writes is the same: targets, prerequisites, recipes.

A Makefile is a graph: target, prerequisites, recipe. make compares the times of the files and runs only what is out of date.